

TripoBD Security Architecture Report

Comprehensive Gap Analysis, Technical Blueprint & Security Benefit Breakdown

Target System: TripoBD Travel Platform (Django + React)	Document Version: 1.0
Source Spec: Security Features Proposed.pdf	Date: August 2026
Scope: Backend (Django 5.2), Frontend (Vite React), MySQL	Status: Proposed Implementation

Executive Summary:

This document presents an exhaustive security evaluation of the **TripoBD** travel ecosystem based on the *Security Features Proposed* specification. Currently, TripoBD leverages Django REST Framework and React, handling sensitive user PII (national IDs, phone numbers, bank details), tour room communications, booking payments, and AI-assisted trip planning. However, the existing infrastructure lacks critical defense-in-depth mechanisms, leaving it exposed to account takeovers, token theft via XSS, IDOR attacks, API abuse/DoS, unencrypted database storage, and malicious file uploads. Integrating the 9 proposed security controls will elevate TripoBD to enterprise-grade security standards.

1. Detailed Security Analysis by Proposed Feature

Feature 1: Advanced Two-Factor Authentication (2FA) with Time-Based One-Time Password (TOTP)

Required Tools/Libraries: [pyotp](#), [qrcode](#)

1. Current Gap in TripoBD Project:

TripoBD's current user authentication workflow relies solely on single-factor credentials (username/password or basic 6-digit email OTPs stored in `otp_verifications`). There is no time-based 2FA (TOTP) mechanism integrated for user accounts, service providers, or administrators. If a user's password is stolen via credential stuffing, phishing, or password reuse, an attacker can gain full access to traveler profiles, service provider verification portals, and booking management without any secondary challenge.

2. Implementation Strategy in TripoBD:

Backend (Django):

- Install packages: `pip install pyotp qrcode Pillow`
- Model Update: Add `totp_secret` (encrypted CharField) and `is_2fa_enabled` (BooleanField) to `AccountSettings` model in `api/models.py`.
- Endpoint Creation in `api/views.py`:
 1. `/api/2fa/setup/`: Generates a secret using `pyotp.random_base32()` and renders a QR code base64 image string with `qrcode.make()` pointing to `otpauth://totp/TripoBD:user@email.com?secret=...`
 2. `/api/2fa/enable/`: Verifies initial 6-digit TOTP token using `totp.verify(code)` before enabling 2FA.
 3. Login Flow Modification: In `CustomTokenObtainPairView`, if `is_2fa_enabled` is `True`, return a pre-auth token requiring second-step verification at `/api/2fa/verify-login/`.

Frontend (React):

- Add a 2FA Management UI under User Profile / Account Settings with QR code rendering for scanning via Google Authenticator or Authy.
- Add a 6-digit TOTP input modal step during the login workflow.

3. Expected Security Benefits:

- Mitigates password-based account takeovers and credential stuffing attacks by 99.9%.
- Guarantees secondary proof-of-possession verification for sensitive roles (Service Providers & System Admins).
- Protects travel funds, booking modifications, and identity document access from stolen password hazards.

Feature 2: Hardened JSON Web Token (JWT) Authentication with HttpOnly Cookies and Refresh Token Rotation

Required Tools/Libraries: [djangorestframework-simplejwt](#)

1. Current Gap in TripoBD Project:

Currently, TripoBD returns JWT access and refresh tokens directly in the HTTP JSON response body upon login. The React frontend stores these tokens in client-side storage (`localStorage` or `sessionStorage`). Any Cross-Site Scripting (XSS) vulnerability or malicious third-party script can trivially read `localStorage.getItem('access_token')` and exfiltrate credentials. Furthermore, refresh tokens are static and not blacklisted or rotated upon use, allowing stolen refresh tokens to be reused indefinitely.

2. Implementation Strategy in TripoBD:

Backend (Django):

- Install package: `pip install djangorestframework-simplejwt`
- Configure Backend/config/settings.py:

```
SIMPLE_JWT = {'ROTATE_REFRESH_TOKENS': True, 'BLACKLIST_AFTER_ROTATION': True,
'ACCESS_TOKEN_LIFETIME': timedelta(minutes=15), 'REFRESH_TOKEN_LIFETIME':
timedelta(days=7)}
```
- Custom Token View: Create `CookieTokenObtainPairView` that sets the refresh token inside an `HttpOnly`, `Secure`, `SameSite=Lax/Strict` HTTP cookie (`res.set_cookie('refresh_token', token, httponly=True, samesite='Lax')`).

Frontend (React):

- Purge all token storage from `localStorage`.
- Maintain the short-lived access token strictly in JavaScript memory state (e.g. `React Context/State`).
- Configure `Axios` interceptor: On HTTP 401 error, automatically invoke `/api/token/refresh/` which automatically transmits the `HttpOnly` refresh cookie.

3. Expected Security Benefits:

- Completely eliminates token theft via client-side XSS scripts because `HttpOnly` cookies are inaccessible to JavaScript.
- Refresh token rotation ensures that if a refresh token is ever intercepted, its first use immediately invalidates the entire token family (blacklisting compromised sessions).
- Enforces strict session lifecycle control and secure logout capability.

Feature 3: Object-Level Access Control and Insecure Direct Object Reference (IDOR) Prevention

Required Tools/Libraries: DRF Custom Permission Classes, UUIDField (optional)

1. Current Gap in TripoBD Project:

TripoBD's current database models (Booking, TripStory, UserProfile, OpenTourGroup, CommunityPost) rely on sequential auto-incrementing integer IDs (`id = 1, 2, 3...`). Furthermore, several API endpoints fetch objects directly using request parameters without enforcing strict owner-level checks. An authenticated user can change endpoint parameters from `/api/bookings/10/object/` to `/api/bookings/11/object/` to view or edit another user's private trip details, personal identity numbers, or booking records.

2. Implementation Strategy in TripoBD:

Backend (Django):

- Create custom permission classes in `api/permissions.py`:
 1. `IsOwnerOrReadOnly`: Enforces `obj.user == request.user` or `obj.user_profile.user == request.user` for write/delete actions.
 2. `IsTourRoomMember`: Verifies `TourRoomMembership.objects.filter(room=obj, user=request.user).exists()`.
- Apply permissions across DRF ViewSets: Add `permission_classes = [IsAuthenticated, IsOwnerOrReadOnly]` and override `get_queryset()` to filter records by the requesting user (`self.request.user`).
- UUID Migration: Add a `uuid = models.UUIDField(default=uuid.uuid4, editable=False, unique=True)` attribute to sensitive models (Booking, TourRoom, ServiceProvider) and lookup by UUID in API routes.

3. Expected Security Benefits:

- Prevents unauthorized viewing, modification, or deletion of other travelers' private data (OWASP #1 Vulnerability: Broken Access Control).
- UUIDs obscure system volume and prevent automated scraper enumeration of platform growth metrics.

Feature 4: Rate Limiting and Anti-Brute-Force Attack Protection

Required Tools/Libraries: DRF `AnonRateThrottle`, `UserRateThrottle`, `django-ratelimit` (optional)

1. Current Gap in TripoBD Project:

TripoBD's `settings.py` currently has NO REST framework throttling or rate limiting configured. Authentication endpoints (`/api/login/`, `/api/register/`, password reset, OTP requests) and expensive AI itinerary generation endpoints (Gemini API calls) can be called infinitely without restriction. This leaves TripoBD vulnerable to password brute-forcing, OTP flooding attacks, SMS/Email gateway cost inflation, and DoS attacks on server CPU/AI API budgets.

2. Implementation Strategy in TripoBD:

Backend (Django):

- Configure throttling in `Backend/config/settings.py` under `REST_FRAMEWORK`:
`'DEFAULT_THROTTLE_CLASSES': ['rest_framework.throttling.AnonRateThrottle', 'rest_framework.throttling.UserRateThrottle']`
`'DEFAULT_THROTTLE_RATES': {'anon': '100/day', 'user': '1000/day', 'login': '5/minute', 'otp': '3/minute', 'ai_gen': '10/hour'}`
- Custom Throttles: Create custom classes in `api/throttling.py` (e.g. `LoginRateThrottle`, `AIGenerationRateThrottle`).
- Attach throttling classes directly to sensitive views: `throttle_classes = [LoginRateThrottle]` on authentication and AI endpoints.
- Automatically return HTTP 429 (Too Many Requests) with a `Retry-After` header.

3. Expected Security Benefits:

- Blocks automated credential stuffing and brute-force password guessing tools.
- Prevents API abuse and financial exhaustion of AI services (Google Gemini API credits).
- Ensures high platform availability by throttling malicious traffic spikes.

Feature 5: Audit Logging and Security Monitoring System

Required Tools/Libraries: [django-simple-history](#)

1. Current Gap in TripoBD Project:

TripoBD lacks database modification history tracking and security event logging. If a service provider's verification status (`is_verified`) is altered, booking status is changed, or repeated failed login attempts occur, there is no audit trail showing who performed the change, what fields were updated, or when it occurred. This makes post-incident forensics, accountability enforcement, and fraud detection impossible.

2. Implementation Strategy in TripoBD:

Backend (Django):

- Install package: `pip install django-simple-history`
- Add 'simple_history' to `INSTALLED_APPS` and middleware 'simple_history.middleware.HistoryRequestMiddleware' in `settings.py`.
- Model Integration: Add `history = HistoricalRecords()` to key models in `api/models.py` (UserProfile, ServiceProvider, Booking, AccountSettings).
- Signal Handlers: Connect Django signals (`user_logged_in`, `user_login_failed`) to log IP addresses and user agents into a `SecurityAuditLog` model.
- Admin Interface: Register historical models with Django Admin to allow system administrators to view side-by-side diffs of data changes.

3. Expected Security Benefits:

- Provides complete historical auditability of all critical data modifications (who, what, when).
- Enables rapid forensic investigation and recovery during data corruption or security breaches.
- Ensures compliance with legal data governance and financial audit standards.

Feature 6: End-to-End Encrypted Tour Room Messaging Using the Web Cryptography API

Required Tools/Libraries: [Web Crypto API \(AES-GCM\)](#)

1. Current Gap in TripoBD Project:

Tour Room group chat messages (TourRoom discussions) and direct communications between travelers and guides are currently transmitted and saved in plain text in the database. Any server compromise, database leak, or rogue database administrator can read all confidential chat histories, travel schedules, and personal phone numbers shared in chat rooms.

2. Implementation Strategy in TripoBD:

Frontend (React - Web Crypto API):

- Key Generation/Derivation: When a user creates or joins a tour room, generate a symmetric AES-GCM 256-bit room encryption key via `window.crypto.subtle.generateKey()` or derive it from a room passphrase using PBKDF2.
- Outgoing Encryption: Before dispatching a message to the backend via WebSocket or REST API, encrypt the text using `window.crypto.subtle.encrypt({name: 'AES-GCM', iv: randomIv}, key, encodedText)`.
- Incoming Decryption: Upon receiving an encrypted payload, decrypt it locally in the browser using the shared room key.

Backend (Django):

- Update room message schema to store only ciphertext, iv (initialization vector base64), and sender_id.
- Server operates in a Zero-Knowledge mode: Django processes and relays messages without ever possessing the decryption key.

3. Expected Security Benefits:

- Guarantees complete end-to-end privacy for tour group members; messages cannot be read by backend servers or DB eavesdroppers.
- Neutralizes data leak risks: Database dumps reveal only mathematically unbreakable AES-GCM ciphertext.
- Builds trust among high-profile travelers and tour guides sharing sensitive travel logistics.

Feature 7: Content Security Policy (CSP) and HTTP Security Headers Implementation

Required Tools/Libraries: [django-csp](#), [SecurityMiddleware](#)

1. Current Gap in TripoBD Project:

TripoBD's Django configuration currently lacks strict HTTP security headers. Missing directives include Content Security Policy (CSP), HTTP Strict Transport Security (HSTS), X-Content-Type-Options, and X-Frame-Options ('DENY'). Without CSP, malicious scripts injected via user-generated content (e.g. trip story HTML or review comments) can execute in the victim's browser, loading unauthorized external scripts, framing the application (clickjacking), or exfiltrating DOM data.

2. Implementation Strategy in TripoBD:

Backend (Django):

- Install package: `pip install django-csp`
- Add middleware: Add `'csp.middleware.CSPMiddleware'` to `MIDDLEWARE` in `settings.py`.
- Configure CSP directives in `settings.py`:
`CSP_DEFAULT_SRC = ('self',)`
`CSP_SCRIPT_SRC = ('self', 'https://maps.googleapis.com')`
`CSP_STYLE_SRC = ('self', 'unsafe-inline', 'https://fonts.googleapis.com')`
`CSP_IMG_SRC = ('self', 'data:', 'https://res.cloudinary.com')`
- SecurityMiddleware Hardening:
`SECURE_BROWSER_XSS_FILTER = True`
`SECURE_CONTENT_TYPE_NOSNIFF = True`
`X_FRAME_OPTIONS = 'DENY'`
`SECURE_HSTS_SECONDS = 31536000 # 1 year (Production)`
`SECURE_HSTS_INCLUDE_SUBDOMAINS = True`

3. Expected Security Benefits:

- Blocks unauthorized Cross-Site Scripting (XSS) script execution and data exfiltration.
- Prevents Clickjacking attacks by forbidding TripoBD from being embedded inside malicious iframes.
- Stops MIME-type sniffing exploits and enforces encrypted HTTPS communication via HSTS.

Feature 8: Zero-Trust File Upload Validation and Malware Scanning

Required Tools/Libraries: `python-magic`, `Pillow`, `uuid4`, `ClamAV` (optional)

1. Current Gap in TripoBD Project:

TripoBD allows users to upload profile pictures, tour photos, NID scans (`ServiceProvider.nid_scan`), and certifications (`ServiceProvider.certification`). Current upload handling does not validate actual file magic bytes or strip EXIF metadata. An attacker can upload a malicious webshell script disguised with a `.jpg` extension (e.g. `shell.php.jpg`), leading to Remote Code Execution (RCE). Additionally, uploaded photos retain EXIF GPS metadata, exposing exact geographic locations of travelers.

2. Implementation Strategy in TripoBD:

Backend (Django):

- Install packages: `pip install python-magic Pillow clamd`
- Validation Utility in `api/utils/file_validator.py`:
 1. Magic Byte Inspection: Use `python-magic` (`magic.from_buffer(file.read(2048), mime=True)`) to verify actual MIME type against an allowed whitelist (`image/jpeg`, `image/png`, `application/pdf`).
 2. Cryptographic Renaming: Rename all uploaded files to unguessable UUIDs using `uuid.uuid4().hex` before writing to disk/S3.
 3. EXIF Stripping: Process uploaded images with `Pillow` (`Image.open()`), strip EXIF metadata (GPS coords, camera model), and re-save clean image bytes.
 4. Malware Scan: Pass uploaded file stream to `ClamAV` daemon (`clamd.scan_stream()`) to detect virus signatures before file persistence.

3. Expected Security Benefits:

- Eliminates Remote Code Execution (RCE) and webshell execution threats via file upload endpoints.
- Protects user privacy by stripping sensitive GPS geolocation metadata embedded in photo uploads.
- Prevents server file overwrites and path traversal vulnerabilities.

Feature 9: Field-Level Encryption for Personally Identifiable Information (PII)

Required Tools/Libraries: [django-cryptography](#) or [django-fernet-fields](#)

1. Current Gap in TripoBD Project:

TripoBD stores sensitive user PII in plain text inside the MySQL database (tripo_db). Fields like `UserProfile.phone_number`, `UserProfile.national_id`, and `ServiceProvider.bank_account_details` are readable in cleartext. If database backups are leaked, SQL injection occurs, or database administrative credentials are breached, all user identity documents, phone numbers, and financial details are exposed in plain text.

2. Implementation Strategy in TripoBD:

Backend (Django):

- Install package: `pip install django-cryptography` (or `django-fernet-fields`)
- Model Refactoring in `api/models.py`:

```
Import from django_cryptography.fields import encrypt
```

Wrap PII fields:

```
phone_number = encrypt(models.CharField(max_length=15))
national_id = encrypt(models.CharField(max_length=20, blank=True, null=True))
bank_account_details = encrypt(models.TextField())
```

- Encryption Key Management: Store the AES master key in environment variable `FIELD_ENCRYPTION_KEY` in `Backend/.env`.
- Migration Script: Run Django data migration to read existing plain text records, encrypt them, and update database entries.

3. Expected Security Benefits:

- Ensures full protection of sensitive traveler identity and financial data even in the event of a total database breach.
- Achieves compliance with global data protection laws (GDPR, National Data Protection Acts).
- Renders stolen database SQL dumps completely useless to threat actors without the environment key.

2. TripoBD Security Implementation Matrix

Ref #	Proposed Feature	Primary Threat Mitigated	OWASP Category	Priority
F-01	TOTP 2FA Authentication	Credential Stuffing, Account Takeover	A07: Identification & Auth Failures	High
F-02	Hardened JWT (HttpOnly)	Token Theft via XSS, Session Hijack	A01: Broken Access Control	High
F-03	Object Access & IDOR	Unauthorized Data Access & Tampering	A01: Broken Access Control	Critical
F-04	Rate Limiting & Throttling	Brute-Force, API Abuse, Resource DoS	A04: Insecure Design / Rate Limit	Medium
F-05	Audit Logging & Monitoring	Repudiation, Undetected Data Tampering	A09: Security Logging Failures	Medium
F-06	E2E Encrypted Chat	Eavesdropping, DB Leak Chat Exposure	A02: Cryptographic Failures	Normal
F-07	CSP & Security Headers	XSS Execution, Clickjacking, Sniffing	A05: Security Misconfiguration	High
F-08	Zero-Trust File Uploads	Remote Code Execution (RCE), Metadata Leak	A03: Injection / Unrestricted Upload	Critical
F-09	Field-Level PII Encryption	Plaintext Data Breach from DB Dump	A02: Cryptographic Failures	High

Conclusion & Recommended Implementation Roadmap:

By implementing these 9 security controls, TripoBD will achieve robust, multi-layered security defense. It is recommended to execute the implementation in three phases:

- Phase 1 (Immediate - High Risk):** Implement IDOR Prevention (F-03), Hardened JWT with HttpOnly (F-02), and Zero-Trust File Upload Validation (F-08).
- Phase 2 (Core Security):** Deploy TOTP 2FA (F-01), Rate Limiting (F-04), CSP/Security Headers (F-07), and Field-Level PII Encryption (F-09).
- Phase 3 (Monitoring & Advanced Privacy):** Integrate Audit Logging (F-05) and E2E Encrypted Chat (F-06).